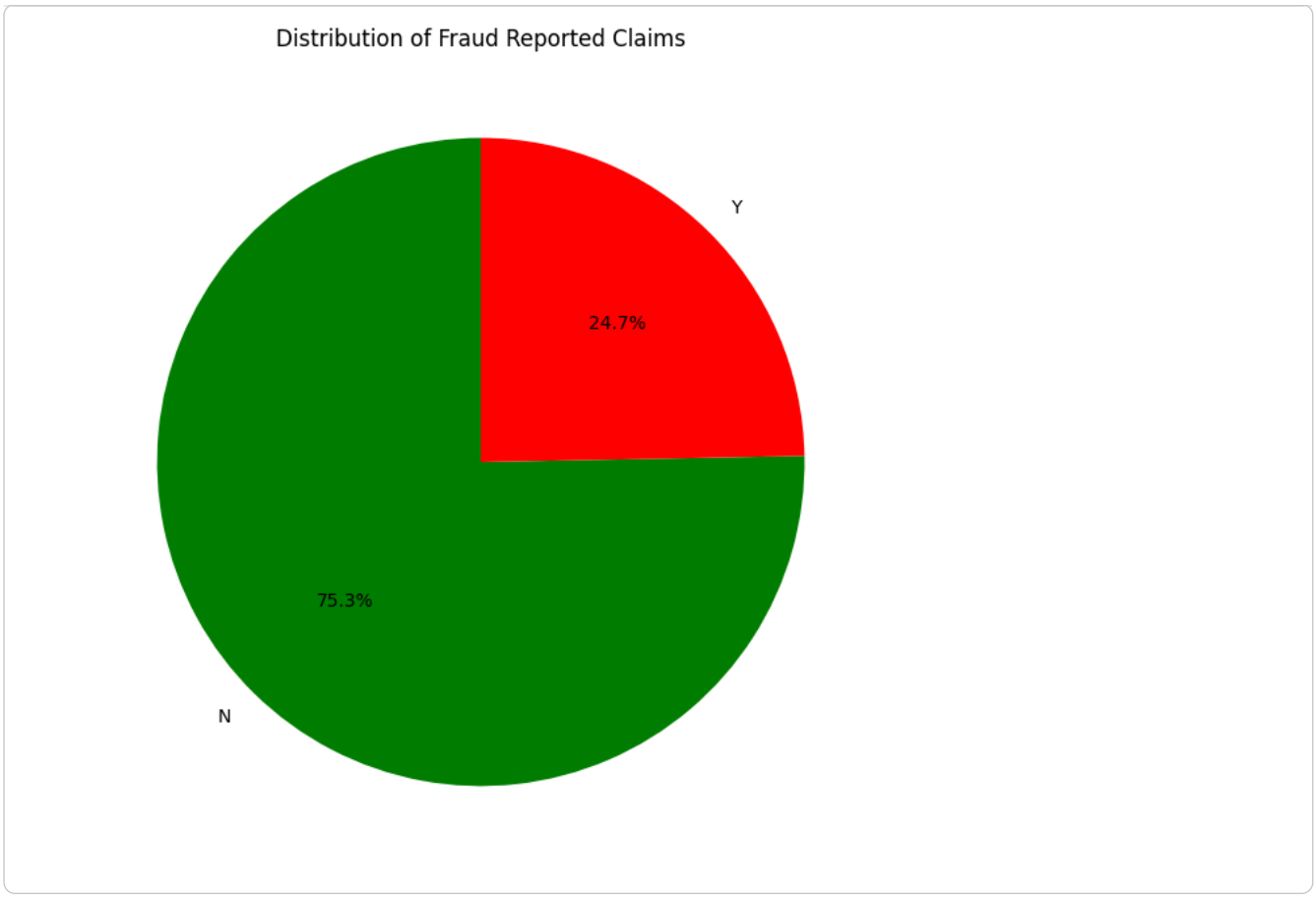
```
import pandas as pd
import numpy as np
import matplotlib as mlt
df = pd.read_csv("/content/insurance_claims.csv")
```

```
df
```

| | months_as_customer | age | policy_number | policy_bind_date | policy_state | policy_csl | policy_deductable | policy_annual_pr |
|---|---|---|---|---|---|---|---|---|
| **0** | 328 | 48 | 521585 | 2014-10-17 | OH | 250/500 | 1000 | 14 |
| **1** | 228 | 42 | 342868 | 2006-06-27 | IN | 250/500 | 2000 | 11 |
| **2** | 134 | 29 | 687698 | 2000-09-06 | OH | 100/300 | 2000 | 14 |
| **3** | 256 | 41 | 227811 | 1990-05-25 | IL | 250/500 | 2000 | 14 |
| **4** | 228 | 44 | 367455 | 2014-06-06 | IL | 500/1000 | 1000 | 15 |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **995** | 3 | 38 | 941851 | 1991-07-16 | OH | 500/1000 | 1000 | 13 |
| **996** | 285 | 41 | 186934 | 2014-01-05 | IL | 100/300 | 1000 | 14 |
| **997** | 130 | 34 | 918516 | 2003-02-17 | OH | 250/500 | 500 | 13 |
| **998** | 458 | 62 | 533940 | 2011-11-18 | IL | 500/1000 | 2000 | 13 |
| **999** | 456 | 60 | 556080 | 1996-11-11 | OH | 250/500 | 1000 | 7 |

1000 rows × 40 columns

```
import matplotlib.pyplot as plt
import seaborn as sns

# Count the occurrences of 'Y' and 'N' in the 'fraud_reported' column
fraud_counts = df['fraud_reported'].value_counts()

# Define custom colors: 'Y' as red, 'N' as green
colors = ['green' if label == 'N' else 'red' for label in fraud_counts.index]

# Create a pie chart with custom colors
plt.figure(figsize=(8, 8))
plt.pie(fraud_counts, labels=fraud_counts.index, autopct='%1.1f%%', startangle=90, colors=colors)
plt.title('Distribution of Fraud Reported Claims')
plt.ylabel('') # Hide the y-label for a cleaner pie chart
plt.show()
```




## Interpretation of Fraud Reported Claims

The pie chart above visualizes the proportion of fraudulent ('Y') versus non-fraudulent ('N') claims in the dataset.

From the chart, we can observe:

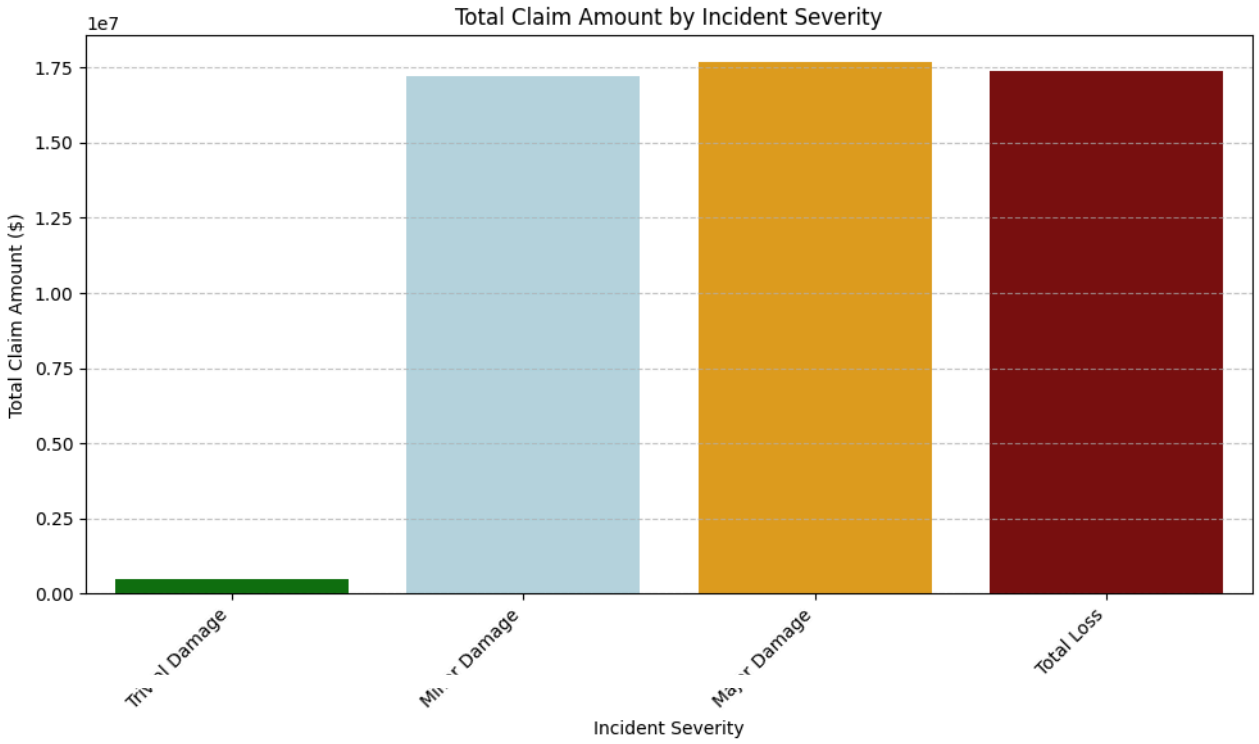
- A significant majority of claims are reported as non-fraudulent, indicated by the larger slice labeled 'N'.
- A smaller, but notable, percentage of claims are reported as fraudulent, indicated by the slice labeled 'Y'.

This distribution highlights that while fraud is present, it constitutes a minority of the total claims, which is typical for insurance datasets.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Group by incident_severity and sum total_claim_amount
claims_by_severity = df.groupby('incident_severity')['total_claim_amount'].sum().reset_index()

# Define custom colors as requested
severity_colors = {
    'Minor Damage': 'lightblue',
    'Major Damage': 'orange',
    'Total Loss': 'darkred',
    'Trivial Damage': 'green'
}

# Sort the claims_by_severity DataFrame based on the custom color order for consistent plotting
# This ensures the colors map correctly even if value_counts changes order or if categories are missing
sort_order = ['Trivial Damage', 'Minor Damage', 'Major Damage', 'Total Loss']
claims_by_severity['incident_severity'] = pd.Categorical(claims_by_severity['incident_severity'], categories=sort_order, or
claims_by_severity = claims_by_severity.sort_values('incident_severity')

# Map the colors to the severity levels for the plot
colors = [severity_colors[severity] for severity in claims_by_severity['incident_severity']]

# Create the bar chart
plt.figure(figsize=(10, 6))
sns.barplot(
    x='incident_severity',
    y='total_claim_amount',
    data=claims_by_severity,
    palette=colors,
    legend=False
)
```

```
plt.title('Total Claim Amount by Incident Severity')
plt.xlabel('Incident Severity')
plt.ylabel('Total Claim Amount ($)')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_15217/757313153.py:26: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(
```




## Interpretation of Total Claim Amount by Incident Severity

The bar chart illustrates the cumulative 'total_claim_amount' for different levels of 'incident_severity'.

From this visualization, we can observe:

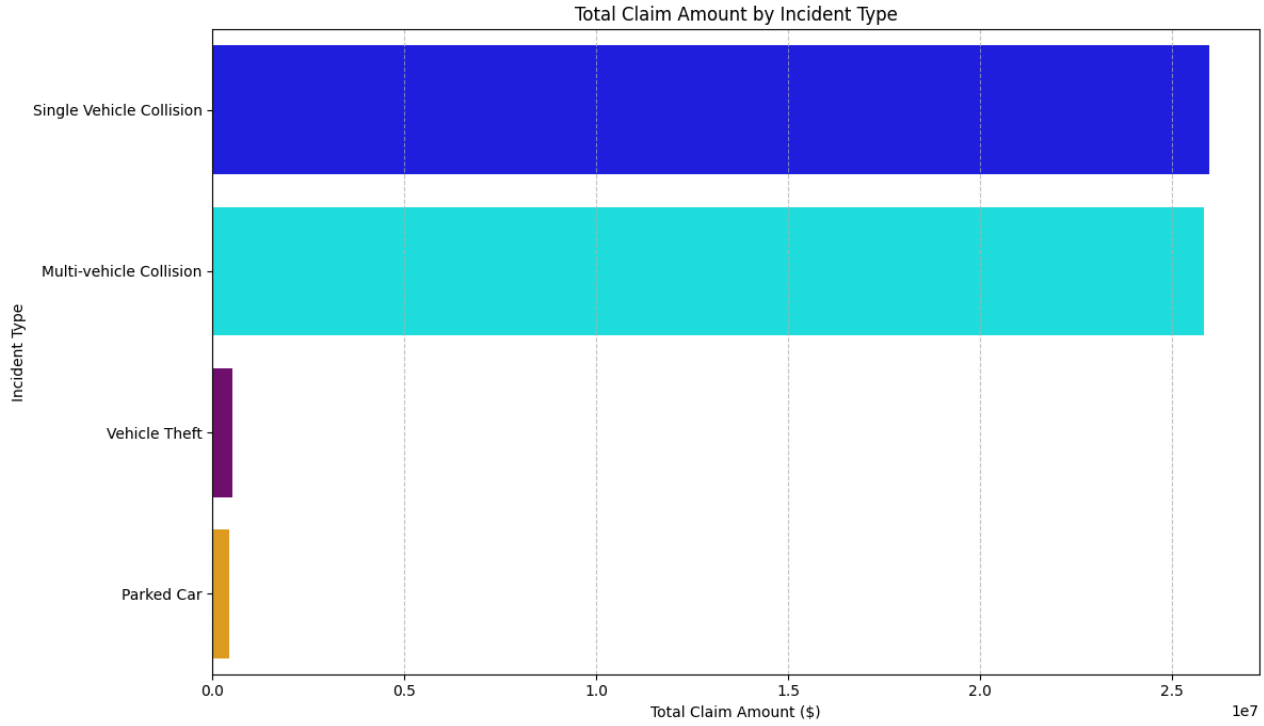
- The **Total Loss** category (dark red) typically accounts for the highest total claim amounts, indicating that these incidents lead to the most significant financial payouts.
- **Major Damage** (orange) also represents a substantial portion of the total claims, usually ranking second in terms of overall cost.
- **Minor Damage** (light blue) and **Trivial Damage** (green) contribute comparatively smaller amounts to the total claims, which is expected given their lower severity.

This breakdown helps in understanding which types of incidents lead to the largest financial burdens, which can be crucial for risk assessment and resource allocation within an insurance company.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Group by incident_type and sum total_claim_amount
claims_by_incident_type = df.groupby('incident_type')['total_claim_amount'].sum().reset_index()

# Sort for better visualization, e.g., by claim amount in descending order
claims_by_incident_type = claims_by_incident_type.sort_values(by='total_claim_amount', ascending=False)

# Define the custom colors
custom_colors = ['blue', 'cyan', 'purple', 'orange', 'teal']

# Ensure there are enough colors for all incident types. If not, cycle or use default for extras.
# For this example, we assume at most 5 unique incident types as per the provided colors.
num_incident_types = len(claims_by_incident_type)
palette_to_use = custom_colors[:num_incident_types]

# Create the horizontal bar chart
plt.figure(figsize=(12, 7))
```

```
sns.barplot(
    x='total_claim_amount',
    y='incident_type',
    data=claims_by_incident_type,
    palette=palette_to_use,
    legend=False
)
plt.title('Total Claim Amount by Incident Type')
plt.xlabel('Total Claim Amount ($)')
plt.ylabel('Incident Type')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_15217/3835552673.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and

  sns.barplot(
```




## Interpretation of Total Claim Amount by Incident Type

The horizontal bar chart illustrates the total financial outlay for each type of incident.

**Direct Interpretation:**

- The incident types are ranked by their total claim amount, showing which types are most costly.
- Incidents like 'Multi-vehicle Collision' and 'Vehicle Theft' likely incur the highest total costs, as these typically involve significant damage or loss.
- Simpler incidents such as 'Parked Car' or 'Rear Collision' generally result in lower overall claim amounts.

```
import plotly.express as px

# Get counts of each auto make
auto_make_counts = df['auto_make'].value_counts().reset_index()
auto_make_counts.columns = ['auto_make', 'num_claims']

# Select top N auto makes (e.g., top 5 to match the number of colors provided)
top_n = 5
top_auto_makes = auto_make_counts.head(top_n)

# Define custom premium colors as requested
premium_colors = ['#000080', '#C0C0C0', '#000000', '#A9A9A9', '#7FDBFF'] # Navy Blue, Silver, Black, Dark Grey, Metallic Cya
```

```
# Create a mapping for specific colors to auto makes to ensure correct assignment
color_map = {
    top_auto_makes.loc[i, 'auto_make']: premium_colors[i]
    for i in range(min(top_n, len(premium_colors)))
}

# Generate the treemap using plotly.express
fig = px.treemap(
    top_auto_makes,
    path=[px.Constant("All Auto Makes"), 'auto_make'], # Add a root node for better treemap structure
    values='num_claims',
    color='auto_make', # Use 'auto_make' to dictate color groups
    color_discrete_map=color_map, # Apply the custom color mapping
    title='Top Auto Makes in Claims'
)

# Update layout for better display
fig.update_layout(margin = dict(t=50, l=25, r=25, b=25))
fig.data[0].textinfo = 'label+value' # Show both label (auto make) and value (number of claims)
fig.show()
```


## Interpretation of Top Auto Makes in Claims Treemap

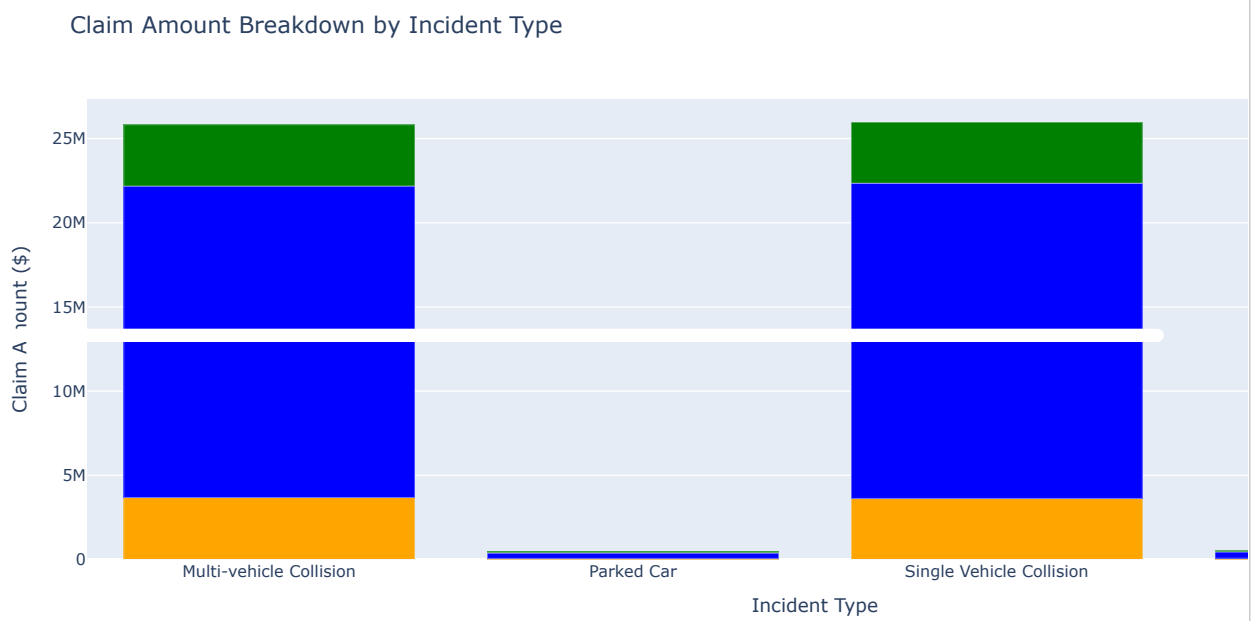
This treemap visually represents the distribution of the top auto makes involved in insurance claims. Each rectangle's size corresponds to the number of claims for that particular auto make, and the premium colors highlight their distinct presence.

**Direct Interpretation:**

- **Dominant Makes:** The largest rectangles (e.g., 'Chevrolet', 'Volkswagen') indicate the auto makes with the highest number of reported claims.
- **Relative Frequency:** The varying sizes of the colored segments quickly show the relative frequency of claims across the top auto makes.
- **Color Coding:** The use of premium colors (Navy Blue, Silver, Black, Dark Grey, Metallic Cyan) helps distinguish between the different auto makes, offering an aesthetically pleasing and informative overview of claims data by manufacturer.

```
import plotly.express as px
import pandas as pd

# Group by incident_type and sum the claim components
claim_breakdown_by_incident = df.groupby('incident_type')[['injury_claim', 'vehicle_claim', 'property_claim']].sum().reset_

# Melt the DataFrame to prepare for stacked bar chart
melted_claims = claim_breakdown_by_incident.melt(id_vars='incident_type',
                                                 value_vars=['injury_claim', 'vehicle_claim', 'property_claim'],
                                                 var_name='claim_type',
                                                 value_name='claim_amount')

# Define custom colors for each claim type as requested
claim_type_colors = {
    'injury_claim': 'orange',
```

```
    'vehicle_claim': 'blue',
    'property_claim': 'green'
}

# Create the stacked column chart using Plotly Express
fig = px.bar(
    melted_claims,
    x='incident_type',
    y='claim_amount',
    color='claim_type',
    color_discrete_map=claim_type_colors,
    title='Claim Amount Breakdown by Incident Type',
    labels={'claim_amount': 'Claim Amount ($)', 'claim_type': 'Claim Type', 'incident_type': 'Incident Type'}
)

fig.update_layout(barmode='stack', xaxis_title='Incident Type', yaxis_title='Claim Amount ($)')
fig.show()
```




## Interpretation of Claim Amount Breakdown by Incident Type

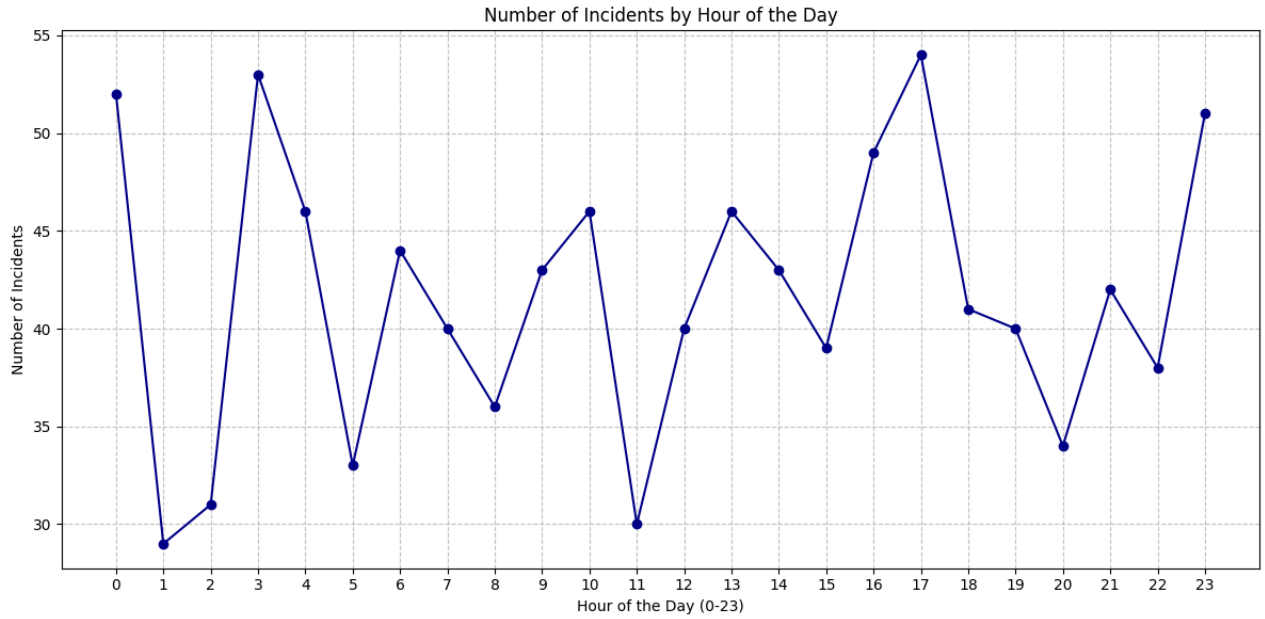
This stacked column chart illustrates how the total claim amount for each incident type is distributed among injury, vehicle, and property damages. Each bar represents an incident type, and the colored segments within it show the proportional contribution of each claim category.

**Direct Interpretation:**

- **Dominant Claim Types:** Observe which claim type (injury, vehicle, or property) constitutes the largest portion of the total claim for a specific incident type. For example, 'Single Vehicle Collision' or 'Multi-vehicle Collision' might have a large 'vehicle_claim' (blue) component.
- **Overall Contribution:** Compare the heights of the stacked bars to understand which incident types result in the highest overall payouts, and how these payouts are broken down.
- **Relative Impact:** This visualization helps identify if certain incident types are predominantly driven by one type of damage (e.g., mostly vehicle damage for minor incidents) or if they involve a more balanced mix of injury, vehicle, and property claims (e.g., for major accidents).

```
import matplotlib.pyplot as plt

# Group by 'incident_hour_of_the_day' and count occurrences
incident_hour_counts = df['incident_hour_of_the_day'].value_counts().sort_index()

# Create the line chart
fig, ax = plt.subplots(figsize=(12, 6), facecolor='white') # Set figure facecolor to white
ax.plot(incident_hour_counts.index, incident_hour_counts.values, color='darkblue', marker='o', linestyle='-')

ax.set_title('Number of Incidents by Hour of the Day')
ax.set_xlabel('Hour of the Day (0-23)')
ax.set_ylabel('Number of Incidents')
ax.set_xticks(range(0, 24)) # Ensure all hours are displayed on the x-axis
ax.grid(True, linestyle='--', alpha=0.7)
```

```
plt.tight_layout()
plt.show()
```




## Interpretation of Incident Hour Analysis

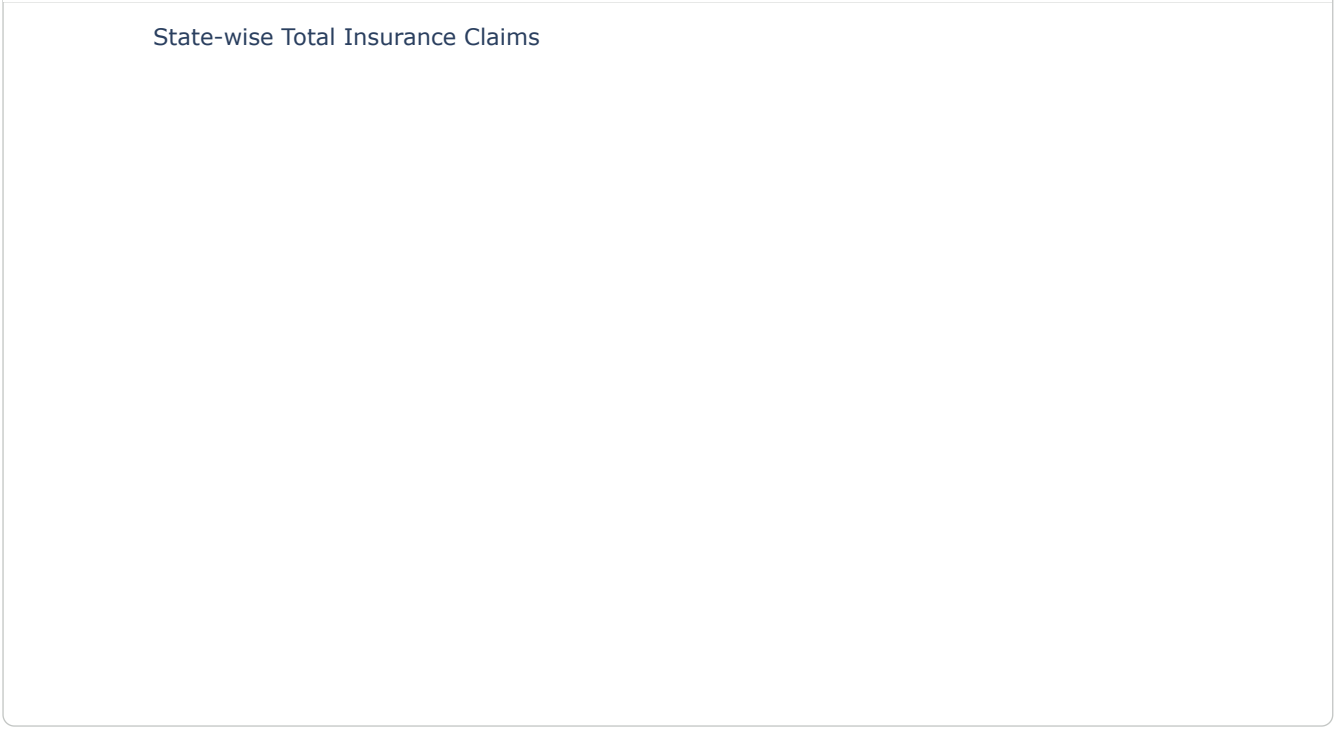
This line chart displays the frequency of incidents across each hour of the day, from midnight (0) to 11 PM (23). It provides a clear visual representation of when incidents are most and least likely to occur.

**Direct Interpretation:**

- **Peak Hours:** Identify the hours with the highest points on the line, indicating the busiest times for incidents. These often correspond to rush hour traffic or periods of increased activity.
- **Off-Peak Hours:** Conversely, the lowest points reveal the hours with the fewest incidents, typically during late-night or early-morning periods.
- **Trends:** Observe any patterns or trends, such as gradual increases or decreases in incidents throughout the morning, afternoon, or evening. This can highlight operational demands at different times of the day.

```
import plotly.express as px

# Aggregate total claim amount by policy state
claims_by_state = df.groupby('policy_state')['total_claim_amount'].sum().reset_index()

# Rename 'policy_state' to 'state_code' for Plotly's built-in state names
claims_by_state = claims_by_state.rename(columns={'policy_state': 'state_code'})

# Define the custom color scale for the gradient
# Light Yellow -> Orange -> Dark Red
custom_color_scale = [
    [0.0, 'rgb(255, 255, 224)'],  # Light Yellow (low claims)
    [0.5, 'rgb(255, 165, 0)'],    # Orange (medium claims)
    [1.0, 'rgb(139, 0, 0)']     # Dark Red (high claims)
]

# Create the choropleth map
fig = px.choropleth(
    claims_by_state,
    locations='state_code',
    locationmode="USA-states", # Specifies that locations are US state abbreviations
    color='total_claim_amount',
    color_continuous_scale=custom_color_scale,
    scope="usa", # Focuses the map on the USA
    title='State-wise Total Insurance Claims',
    labels={'total_claim_amount':'Total Claim Amount ($)'}
)

fig.update_layout(
```

```
    geo_scope='usa',
    margin={"r":0,"t":50,"l":0,"b":0}
)

fig.show()
```



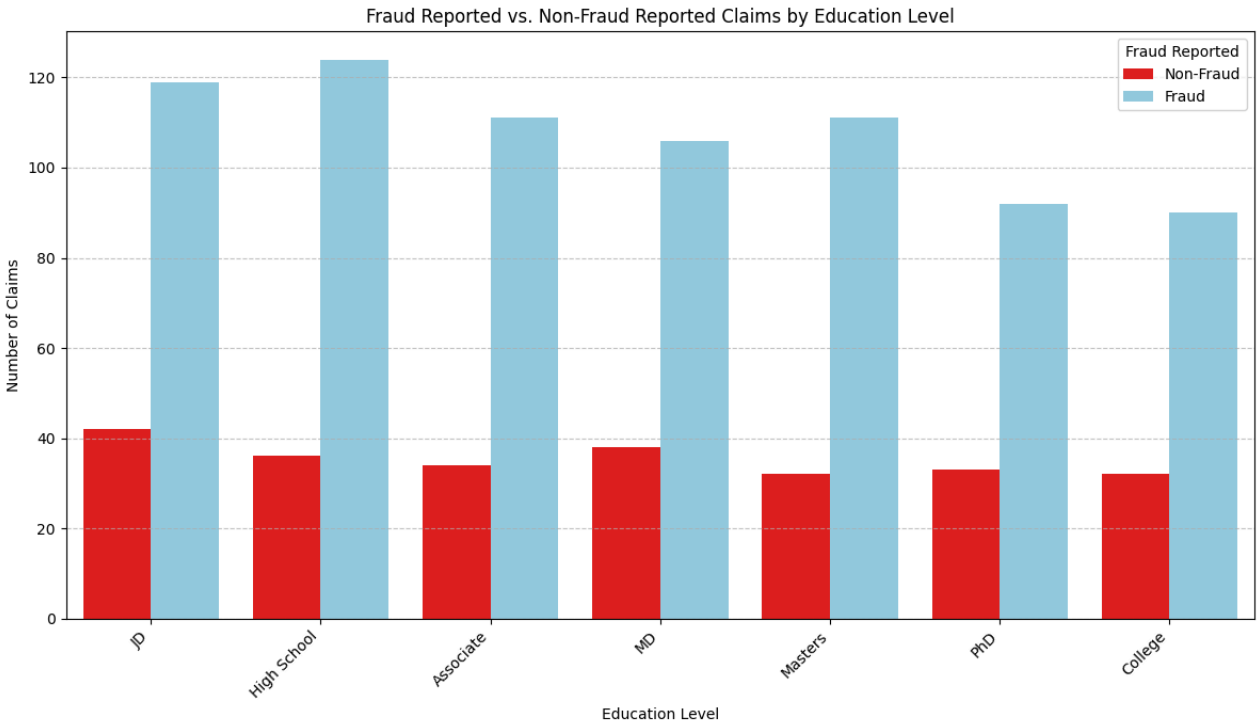

## Interpretation of State-wise Insurance Claims Map

This choropleth map visually represents the total insurance claim amounts across different states in the USA. The color gradient, from light yellow (low claims) to orange (medium claims) and dark red (high claims), allows for quick identification of regions with varying claim activity.

**Direct Interpretation:**

- **High-Claim States:** States shaded in darker red indicate a higher cumulative 'total_claim_amount'. These are regions where insurance companies might experience greater financial exposure due to the volume or severity of incidents.
- **Low-Claim States:** States appearing in lighter yellow suggest lower total claim amounts, potentially indicating fewer incidents or less costly claims within those areas.
- **Geographical Patterns:** The map helps in identifying any geographical concentrations or clusters of high or low claims, which could be influenced by factors like population density, weather patterns, economic activity, or local regulations.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the figure size
plt.figure(figsize=(12, 7))

# Define custom colors: 'N' as blue, 'Y' as red
colors = {'N': 'skyblue', 'Y': 'red'}

# Create the clustered bar chart using seaborn.countplot
sns.countplot(
    x='insured_education_level',
    hue='fraud_reported',
    data=df,
    palette=colors,
    order=df['insured_education_level'].value_counts().index # Order by frequency for better readability
)

plt.title('Fraud Reported vs. Non-Fraud Reported Claims by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Number of Claims')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.legend(title='Fraud Reported', labels=['Non-Fraud', 'Fraud'])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```




## Interpretation of Education Level vs. Fraud Reported Claims

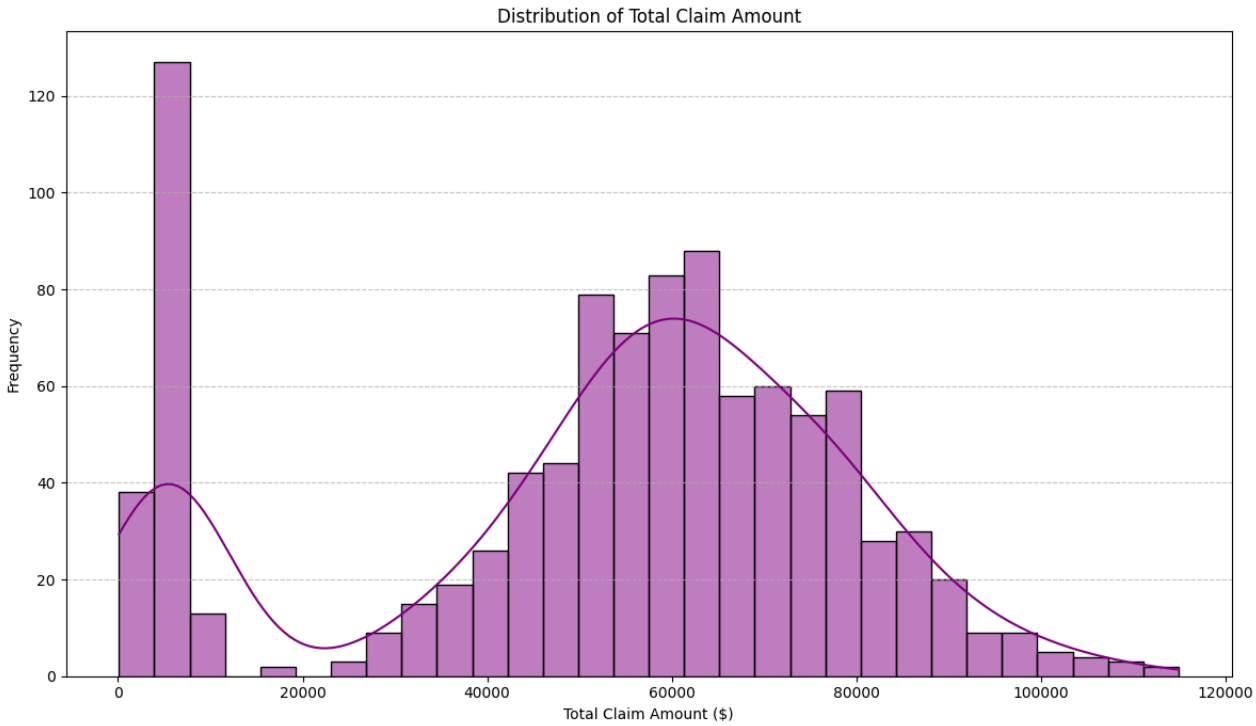
The clustered bar chart visualizes the distribution of fraudulent ('Y') and non-fraudulent ('N') claims across different educational levels of the insured individuals.

**Direct Interpretation:**

- **Overall Claim Volume by Education Level:** The total height of each cluster (blue + red bars) indicates the total number of claims made by individuals within that education level. This helps to understand which education levels are most represented in the dataset's claims.
- **Fraud Proportion within Each Education Level:** By comparing the red (fraudulent) and blue (non-fraudulent) bars within each education level's cluster, we can observe the proportion of fraud. For example, if the red bar is proportionally larger for a specific education level, it might suggest a higher incidence of fraud among that group relative to their total claims.
- **Identification of Patterns:** This chart can help identify if there are any noticeable patterns or correlations between an individual's education level and their propensity to report fraudulent claims. For instance, some education levels might show a higher or lower absolute number of fraudulent claims, or a different fraud-to-non-fraud ratio.

This visualization is useful for understanding demographic aspects related to fraud and can inform targeted risk assessment strategies.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 7))

sns.histplot(df['total_claim_amount'], bins=30, kde=True, color='purple')

plt.title('Distribution of Total Claim Amount')
plt.xlabel('Total Claim Amount ($)')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```




## Interpretation of Total Claim Amount Distribution

The histogram visualizes the frequency distribution of 'total_claim_amount' across all claims in the dataset.

**Direct Interpretation:**

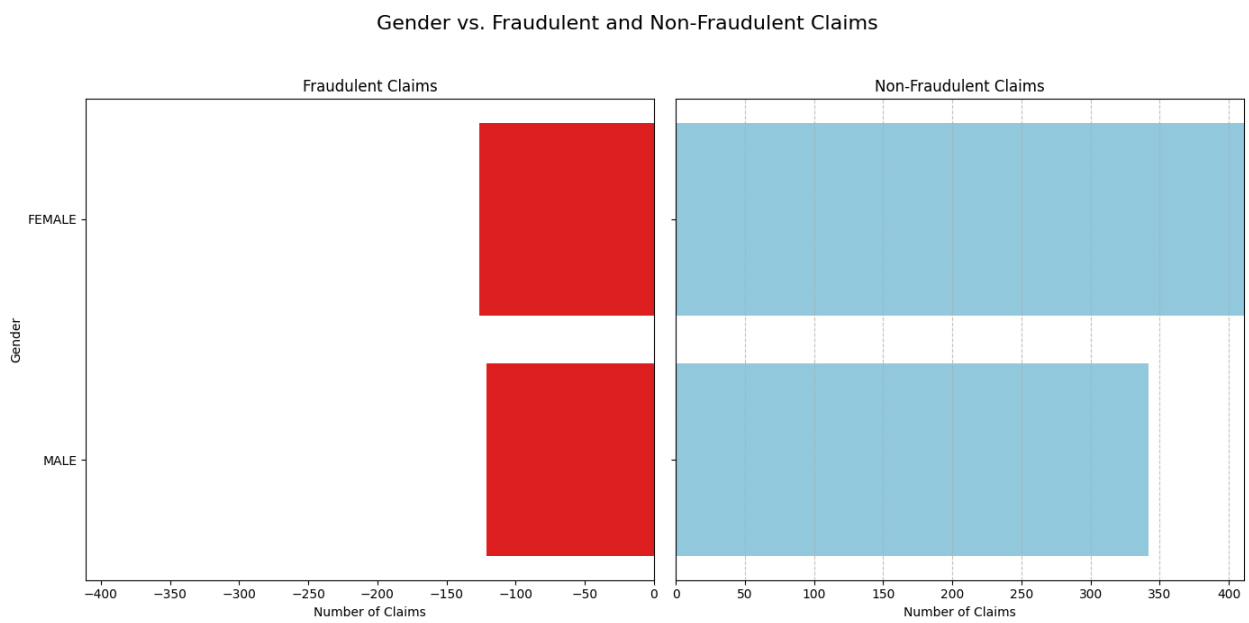
- **Shape of Distribution:** Observe the overall shape of the histogram. It might be skewed (e.g., right-skewed, meaning a long tail to the right, indicating more smaller claims and fewer very large claims) or approximately normal.
- **Central Tendency:** The peak(s) of the histogram indicate the most common ranges of claim amounts.
- **Range of Claims:** The x-axis shows the minimum and maximum claim amounts, giving an idea of the spread of financial values involved in claims.
- **Outliers:** Very high claim amounts, far from the main body of the distribution, might represent outlier events that warrant further investigation.

This distribution helps in understanding the typical financial impact of claims and identifying any unusual patterns or extreme values that could influence risk assessment and financial planning.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Prepare the data
gender_fraud_counts = df.groupby(['insured_sex', 'fraud_reported']).size().unstack(fill_value=0)

# Extract data for plotting
genders = gender_fraud_counts.index
non_fraud_counts = gender_fraud_counts['N']
fraud_counts = gender_fraud_counts['Y']

# Create the figure and a set of subplots
fig, axes = plt.subplots(ncols=2, figsize=(14, 7), sharey=True)

# Plotting Non-Fraudulent Claims (right side, blue)
sns.barplot(x=non_fraud_counts, y=genders, ax=axes[1], palette=['skyblue'], order=genders)
axes[1].set_title('Non-Fraudulent Claims')
axes[1].set_xlabel('Number of Claims')
axes[1].tick_params(axis='x', rotation=0)
axes[1].grid(axis='x', linestyle='--', alpha=0.7)

# Plotting Fraudulent Claims (left side, red)
```

```
# Plot as negative values to create the butterfly effect
sns.barplot(x=-fraud_counts, y=genders, ax=axes[0], palette=['red'], order=genders)
axes[0].set_title('Fraudulent Claims')
axes[0].set_xlabel('Number of Claims')
axes[0].tick_params(axis='x', rotation=0)
# Adjust x-axis limits to match the other plot for better comparison
max_claims = max(non_fraud_counts.max(), fraud_counts.max())
axes[0].set_xlim(-max_claims, 0)
axes[1].set_xlim(0, max_claims)

# Remove y-axis label for the left plot as it's shared
axes[0].set_ylabel('Gender')

# Set the overall title
fig.suptitle('Gender vs. Fraudulent and Non-Fraudulent Claims', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95]) # Adjust layout to prevent suptitle overlap
```

```
/tmp/ipykernel_15217/3855777.py:17: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` a


/tmp/ipykernel_15217/3855777.py:17: UserWarning:


The palette list has fewer values (1) than needed (2) and will cycle, which may produce an uninterpretable plot.

/tmp/ipykernel_15217/3855777.py:25: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` a


/tmp/ipykernel_15217/3855777.py:25: UserWarning:


The palette list has fewer values (1) than needed (2) and will cycle, which may produce an uninterpretable plot.
```




Interpretation of Gender vs. Fraudulent and Non-Fraudulent Claims (Butterfly Chart)